

LAPORAN PRATIKUM PEMROGRAMAN

PRATIKUM 4

STATEMENT PENENTU KEPUTUSAN



Disusun Oleh :

NAMA	: Muhammad Aditya Rahmansyah
NRP	: 3226600122
KELAS	: ST.r Tekkom A
MATA KULIAH	: Pratikum Pemrograman
DOSEN	: Ir. Sigit Wasista, M.Kom.
ASSISTEN DOSEN	: Rofiulloh Mu'azin Al-insof, A.Md.
TANGGAL PRATIKUM	: Rabu, 23 September 2026

PROGRAM STUDI SARJANA TERAPAN TEKNIK KOMPUTER
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026

1. TUJUAN PRAKTIKUM

Setelah melaksanakan Praktikum 4, mahasiswa diharapkan mampu:

1. Menjelaskan konsep statement penentu keputusan dalam program.
 2. Menggunakan statement `if` dan `if-else`.
 3. Menggunakan struktur `else-if` dan `nested if`.
 4. Menggunakan compound statement.
 5. Menggunakan `switch-case`.
 6. Menggunakan operator relasional dan operator logika dalam pembentukan kondisi.
 7. Menguji program dengan berbagai data masukan dan menganalisis hasil eksekusinya.
-

2. DASAR TEORI

Jelaskan secara singkat konsep:

1. Statement Penentu Keputusan

Statement yang membuat program memilih jalur eksekusi berdasarkan suatu kondisi. Di C: `if`, `if-else`, `else-if`, dan `switch-case`.

2. Operator Relasional

Operator pembandingan dua nilai. Hasilnya benar (1) atau salah (0): `==`, `!=`, `>`, `<`, `>=`, `<=`.

3. Operator Logika

Operator penggabung beberapa kondisi:

- `&&` (AND): benar jika semua kondisi benar.
- `||` (OR): benar jika minimal satu kondisi benar.
- `!` (NOT): membalik nilai kebenaran.
- Contoh: `(umur >= 17 && umur <= 60)`.

4. `if`

Menjalankan blok hanya jika kondisi benar.

Contoh :

```
if (nilai >= 60) {  
    printf("Lulus");  
}
```

5. `if-else`

Menyediakan dua jalur: satu jika kondisi benar, satu jika salah.

Contoh :

```
if (x % 2 == 0) printf("Genap");
```

```
else printf("Ganjil");
```

6. `else-if`

Rangkaian pengecekan untuk lebih dari dua kemungkinan. Kondisi dicek berurutan dan berhenti di yang pertama benar.

Contoh:

```
if (n >= 80) printf("A");
```

```
else if (n >= 70) printf("B");
```

```
else printf("C");
```

7. `nested if`

`if` di dalam `if` lain, dipakai untuk pengecekan bertingkat.

Contoh:

```
if (x > 0) {  
    if (x % 2 == 0) printf("Positif genap");  
}
```

8. **Compound Statement**

Sekumpulan statement dalam kurung kurawal { } yang diperlakukan sebagai satu kesatuan. Berguna saat `if/else` perlu menjalankan lebih dari satu perintah.

Contoh :

```
if (D > 0) {  
    x1 = ...;  
    x2 = ...;  
}
```

9. `switch-case`

Memilih satu dari banyak pilihan berdasarkan nilai satu variabel (integer atau karakter). `break` menghentikan eksekusi agar tidak lanjut ke case berikutnya, dan `default` menangani nilai di luar semua case.

Contoh :

```
switch (op) {  
    case '+': hasil = a + b; break;  
    case '-': hasil = a - b; break;  
    default: printf("Tidak valid");  
}
```

Catatan: Tidak perlu menyalin seluruh teori dari modul. Cukup jelaskan pengertian, fungsi, dan contoh sederhana.

3. PERCOBAAN

Percobaan 1 – Statement `if`

1. Tujuan Percobaan

Program ini bertujuan menentukan nilai absolut suatu bilangan

2. Kode Program

```
#include <stdio.h>  
  
int main(void) {  
  
    int bil, absolut;  
  
    printf("Masukkan bilangan bulat: ");  
    scanf("%d", &bil);  
  
    absolut = bil;
```

```

if (bil < 0)

absolut = -bil;

printf("Nilai absolut dari %d adalah %d\n", bil, absolut);

return 0;
}

```

3. Hasil Running

```

Masukkan bilangan bulat: 6
Nilai absolut dari 6 adalah 6

```

```

Masukkan bilangan bulat: -6
Nilai absolut dari -6 adalah 6

```

4. Analisis

Jelaskan:

- Input yang diberikan. Program menerima input berupa satu bilangan bulat (integer) dari user, disimpan dalam variabel `bil`. Bilangan ini bisa berupa angka positif, negatif, atau nol — contohnya di soal: 6 dan -6.
- Kondisi yang diperiksa. Program memeriksa apakah bilangan yang diinput lebih kecil dari (`bil < 0`), yaitu mengecek apakah bilangan tersebut negatif atau tidak.
- Proses yang dilakukan ketika kondisi benar.
Jawaban : Ketika `bil < 0` benar, statement di bawah `if` dijalankan: `absolut = -bil`;
- Output yang dihasilkan.
Jawaban : Program selalu mencetak satu baris: Nilai absolut dari `<bil>` adalah `<absolut>`
- Hubungan antara kondisi dan output.

Jawaban :

Kondisi `bil < 0` menentukan apakah nilai absolut diubah atau tidak sebelum dicetak:

- **Kondisi benar (bil negatif):** absolut dibalik tandanya, sehingga output menampilkan nilai positif yang berbeda dari input. Contoh: -6 menjadi 6.
- **Kondisi salah (bil nol atau positif):** absolut tidak berubah, sehingga output menampilkan angka yang sama dengan input. Contoh: 6 tetap 6.

Percobaan 2 – `if-else` dan Operator Modulus

1. Kode Program

```
#include <stdio.h>

int main(void) {

    int pembilang, penyebut, sisa;

    printf("Masukkan pembilang: ");
    scanf("%d", &pembilang);

    printf("Masukkan penyebut: ");
    scanf("%d", &penyebut);

    if (penyebut == 0) {
        printf("Penyebut tidak boleh 0.\n");
    } else {

        sisa = pembilang % penyebut;

        if (sisa != 0)
            printf("%d tidak habis dibagi %d.\n", pembilang, penyebut);
        else
            printf("%d habis dibagi %d.\n", pembilang, penyebut);
    }

    return 0;
}
```

2. Hasil Running

```
Masukkan pembilang: 10
Masukkan penyebut: 5
10 habis dibagi 5.
```

```
Masukkan pembilang: 7
Masukkan penyebut: 0
Penyebut tidak boleh 0.
```

```
Masukkan pembilang: 10
Masukkan penyebut: 3
10 tidak habis dibagi 3.
```

3. Analisis

Jelaskan penggunaan:

- if-else Statement if-else digunakan ketika terdapat dua kemungkinan proses: satu proses dijalankan jika kondisi benar dan proses lainnya jika kondisi salah.
- nested if Nested if adalah penggunaan statement if di dalam statement if lainnya
- operator % sisa bagi atau modulus
- kondisi penyebut == 0. Digunakan untuk memeriksa apakah nilai penyebut sama dengan 0.

Percobaan 3 – Kondisi pada Karakter

1. Kode Program

```
#include <stdio.h>

int main(void) {

    char c;

    printf("Masukkan suatu karakter: ");
    scanf(" %c", &c);

    if (c >= 'a' && c <= 'z')

        printf("Karakter tersebut termasuk huruf kecil.\n");
    else
```

```
printf("Karakter tersebut bukan huruf kecil.\n");
```

```
return 0;
```

```
}
```

2. Hasil Running

```
Masukkan suatu karakter: a
Karakter tersebut termasuk huruf kecil.
Masukkan suatu karakter: A
Karakter tersebut bukan huruf kecil.
```

3. Analisis

Jelaskan penggunaan operator:

&&

Digunakan untuk menggabungkan dua kondisi. Hasilnya bernilai benar jika kedua kondisi terpenuhi.

serta kondisi:

```
c >= 'a' && c <= 'z'
```

Kondisi ini digunakan untuk memeriksa apakah karakter c termasuk huruf kecil. Karakter harus memenuhi dua kondisi sekaligus, yaitu c lebih besar atau sama dengan 'a' dan c lebih kecil atau sama dengan 'z'.

Percobaan 4 – Demonstrasi goto

1. Kode Program

```
#include <stdio.h>
```

```
int main(void) {
```

```
    int i = 0, hasil = 1;
```

```
    ulang:
```

```
    i++;
```

```

    hasil *= 2;
    printf("Iterasi ke-%2d: %5d\n", i, hasil);
    if (i < 10)
        goto ulang;

    return 0;

}

```

2. Hasil Running

```

Iterasi ke- 1:      2
Iterasi ke- 2:      4
Iterasi ke- 3:      8
Iterasi ke- 4:     16
Iterasi ke- 5:     32
Iterasi ke- 6:     64
Iterasi ke- 7:    128
Iterasi ke- 8:    256
Iterasi ke- 9:    512
Iterasi ke-10:  1024

```

3. Analisis Jelaskan:

- fungsi label ulang:
digunakan sebagai penanda atau titik tujuan yang dapat dituju kembali oleh statement goto
- fungsi goto
digunakan untuk memindahkan jalannya program ke label tertentu, contohnya pada program diatas goto ulang: membuat program kembali ke bagian ulang.
- kondisi $i < 10$
digunakan untuk menentukan apakah pengulangan masih dilanjutkan. Jika i masih kurang dari 10, maka program menjalankan goto ulang: Jika i sudah mencapai 10, program goto tidak dijalankan karena sudah memenuhi kondisi.
- bagaimana program melakukan pengulangan.

Program melakukan pengulangan dengan cara meningkatkan nilai i menggunakan $i++$, menghitung hasil $\times 2$, kemudian mencetak hasilnya. Setelah itu, program memeriksa kondisi $i < 10$. Jika kondisi benar, goto ulang; mengembalikan program ke label ulang:. Proses tersebut terus dilakukan sampai nilai i mencapai 10.

Percobaan 5 – Compound Statement

1. Kode Program

```
#include <stdio.h>

int main(void) {

    char tag;

    printf("Masukkan tipe tag (2 / 4): ");
    scanf(" %c", &tag);

    if (tag == '2') {
        printf("Merek sepeda motor bertipe 2 tag:\n");
        printf("1. YAMAHA\n");
        printf("2. SUZUKI\n");
        printf("3. HONDA\n");
    } else {
        printf("Merek sepeda motor bertipe 4 tag:\n");
        printf("1. HONDA\n");
        printf("2. SUZUKI\n");
        printf("3. YAMAHA\n");
    }
    return 0;
}
```

2. Hasil Running

```
Masukkan tipe tag (2 / 4): 2
Merek sepeda motor bertipe 2 tag:
1. YAMAHA
2. SUZUKI
3. HONDA
Masukkan tipe tag (2 / 4): 4
Merek sepeda motor bertipe 4 tag:
1. HONDA
2. SUZUKI
3. YAMAHA
```

3. Analisis

Jelaskan mengapa beberapa statement `printf()` perlu dikelompokkan menggunakan `{ }`.

Karena ada beberapa statement `printf()` yang harus dijalankan sebagai satu kelompok berdasarkan kondisi `if` atau `else`. Tanda `{ }` digunakan untuk mengelompokkan beberapa statement agar semuanya dijalankan ketika kondisi terpenuhi.

Percobaan 6 – `else-if`

1. Kode Program

```
#include <stdio.h>

int main(void) {

    char operator;
    float bil1, bil2, hasil;

    printf("Masukkan: bilangan1 operator bilangan2\n");
    printf("Operator: + - * /\n");
    printf("Input: ");
    scanf("%f %c %f", &bil1, &operator, &bil2);

    if (operator == '*')
        hasil = bil1 * bil2;
    else if (operator == '/')
        hasil = bil1 / bil2;
    else if (operator == '+')
        hasil = bil1 + bil2;
    else if (operator == '-')
        hasil = bil1 - bil2;
    else {
        printf("Operator tidak dikenali.\n");
    }
}
```

```

return 0;

}

printf("Hasil: %.2f %c %.2f = %.2f\n",
bil1, operator, bil2, hasil);

return 0;

}

```

2. Hasil Running

```

Masukkan: bilangan1 operator bilangan2
Operator: + - * /
Input: 5+5
Hasil: 5.00 + 5.00 = 10.00
Masukkan: bilangan1 operator bilangan2
Operator: + - * /
Input: 10-5
Hasil: 10.00 - 5.00 = 5.00
Masukkan: bilangan1 operator bilangan2
Operator: + - * /
Input: 5*2
Hasil: 5.00 * 2.00 = 10.00
Masukkan: bilangan1 operator bilangan2
Operator: + - * /
Input: 100/20
Hasil: 100.00 / 20.00 = 5.00

```

3. Analisis

Jelaskan:

- input bilangan dan operator;
Program menerima input berupa dua bilangan dan satu operator melalui scanf(). Bilangan disimpan dalam variabel bil1 dan bil2, sedangkan operator disimpan dalam variabel op. Input yang dimasukkan berbentuk bilangan pertama, operator, kemudian bilangan kedua
- urutan pemeriksaan kondisi;
Pemeriksaan kondisi dilakukan secara berurutan menggunakan if dan else-if. Program pertama memeriksa apakah op adalah *, kemudian /, +, dan terakhir -. Pemeriksaan akan berhenti ketika ditemukan kondisi yang sesuai dengan operator yang dimasukkan.
- fungsi else-if;
Fungsi else-if adalah memberikan kondisi alternatif yang akan diperiksa apabila kondisi sebelumnya tidak terpenuhi. Dengan demikian, program dapat menangani beberapa kemungkinan operator dalam satu rangkaian percabangan.

- bagaimana program menentukan operasi yang dijalankan.
Program menentukan operasi yang dijalankan dengan membandingkan nilai op dengan karakter operator. Jika op adalah *, program melakukan perkalian. Jika /, program melakukan pembagian. Jika +, program melakukan penjumlahan, sedangkan jika -, program melakukan pengurangan. Jika operator yang dimasukkan tidak sesuai dengan kondisi yang tersedia, program menampilkan pesan “Operator tidak dikenali.”
-

Percobaan 7 – switch-case

1. Kode Program

```
#include <stdio.h>

int main(void) {

    char op;

    float bil1, bil2, hasil;

    printf("Masukkan: bilangan1 operator bilangan2\n");
    printf("Operator: + - * /\n");
    printf("Input: ");
    scanf("%f %c %f", &bil1, &op, &bil2);

    switch (op) {
        case '*':
            hasil = bil1 * bil2;
            break;
        case '/':
            if (bil2 == 0) {
                printf("Pembagian dengan nol tidak diperbolehkan.\n");
                return 0;
            }

            hasil = bil1 / bil2;
            break;
        case '+':
            hasil = bil1 + bil2;
            break;
        case '-':
            hasil = bil1 - bil2;
            break;

        default:
            printf("Operator tidak dikenali.\n");
    }
```

```

return 0;
}
printf("Hasil: %.2f %c %.2f = %.2f\n",
bil1, op, bil2, hasil);
return 0;
}

```

2. Hasil Running

```

Masukkan: bilangan1 operator bilangan2
Operator: + - * /
Input: 10+4
Hasil: 10.00 + 4.00 = 14.00
Masukkan: bilangan1 operator bilangan2
Operator: + - * /
Input: 10-4
Hasil: 10.00 - 4.00 = 6.00
Masukkan: bilangan1 operator bilangan2
Operator: + - * /
Input: 10*4
Hasil: 10.00 * 4.00 = 40.00
Masukkan: bilangan1 operator bilangan2
Operator: + - * /
Input: 40/10
Hasil: 40.00 / 10.00 = 4.00
Masukkan: bilangan1 operator bilangan2
Operator: + - * /
Input: 1/0
Pembagian dengan nol tidak diperbolehkan.
Masukkan: bilangan1 operator bilangan2
Operator: + - * /
Input: 10]4
Operator tidak dikenali.

```

3. Analisis Jelaskan:

- fungsi `switch`;
Fungsi `switch` digunakan untuk memilih operasi berdasarkan nilai yang terdapat pada variabel `op`. Program akan mencocokkan nilai `op` dengan beberapa pilihan case yang tersedia.
- fungsi `case`;
Fungsi `case` digunakan untuk menentukan operasi yang akan dijalankan sesuai dengan operator yang dimasukkan. Pada program ini terdapat case untuk perkalian (*), pembagian (/), penjumlahan (+), dan pengurangan (-).
- fungsi `break`;

Fungsi break digunakan untuk menghentikan proses pada switch setelah case yang sesuai selesai dijalankan. Dengan adanya break, program tidak melanjutkan ke case berikutnya.

- `fungsi default;`
Fungsi default digunakan ketika nilai `op` tidak sesuai dengan semua case yang tersedia. Program kemudian menampilkan pesan “Operator tidak dikenali.” dan menghentikan program.
- penanganan pembagian dengan nol.

Pada pembagian dengan nol, program melakukan pengecekan `if (bil2 == 0)` sebelum melakukan pembagian. Jika `bil2` bernilai nol, program menampilkan pesan “Pembagian dengan nol tidak diperbolehkan.” kemudian menghentikan program dengan `return 0`. Jika `bil2` bukan nol, pembagian dapat dilakukan.

Ketujuh percobaan tersebut memang merupakan bagian dari modul Praktikum 4.

E. TABEL PENGUJIAN PROGRAM

Setiap program sebaiknya diuji menggunakan beberapa kondisi, termasuk **data normal**, **kondisi batas**, dan **data yang dapat menyebabkan kesalahan**.

No	Data uji/input	Kondisi yang diuji	Hasil yang diharapkan	Hasil aktual	Keterangan
1	10 + 5	Kondisi normal	15.00	15.00	sesuai
2	10[4	Operator tidak valid	Operator tidak dikenali.	Operator tidak dikenali.	sesuai
3	1/0	Pembagian dengan 0	"Pembagian dengan nol	"Pembagian dengan nol	sesuai

			tidak diperbolehkan."	tidak diperbolehkan."	
4	40/10	Pembagian normal	4	4	sesuai
5	10%3	modulus	10 tidak habis dibagi 3	10 tidak habis dibagi 3	sesuai

F. ANALISIS HASIL PRAKTIKUM

Buat analisis secara keseluruhan.

Berdasarkan praktikum yang telah dilakukan, seluruh program dapat dijalankan dengan baik dan benar. Statement penentu keputusan digunakan untuk mengatur alur program berdasarkan kondisi tertentu. Penggunaan if, if-else, dan else-if dapat digunakan untuk menangani berbagai kondisi, sedangkan switch-case digunakan untuk memilih proses berdasarkan nilai tertentu. Operator relasional dan logika juga digunakan untuk membentuk kondisi yang diperiksa oleh program.

Dari ketujuh percobaan, program dapat menentukan proses yang dijalankan berdasarkan data masukan. Pada percobaan if dan if-else, program memeriksa kondisi sebelum menentukan statement yang dijalankan. Pada percobaan kondisi karakter, operator && digunakan untuk memastikan dua kondisi terpenuhi. Compound statement digunakan untuk mengelompokkan beberapa statement agar dapat dijalankan dalam satu kondisi.

Selain itu, percobaan goto menunjukkan bahwa alur program dapat dikembalikan ke label tertentu selama kondisi yang ditentukan masih terpenuhi. Pada else-if dan switch-case, program dapat memilih salah satu proses berdasarkan operator yang diberikan. Pada switch-case, break digunakan untuk menghentikan proses pada case yang sesuai, sedangkan default menangani operator yang tidak tersedia. Pembagian dengan nol juga ditangani secara khusus agar program tidak melakukan operasi yang tidak diperbolehkan.

G. JAWABAN PERTANYAAN ANALISIS

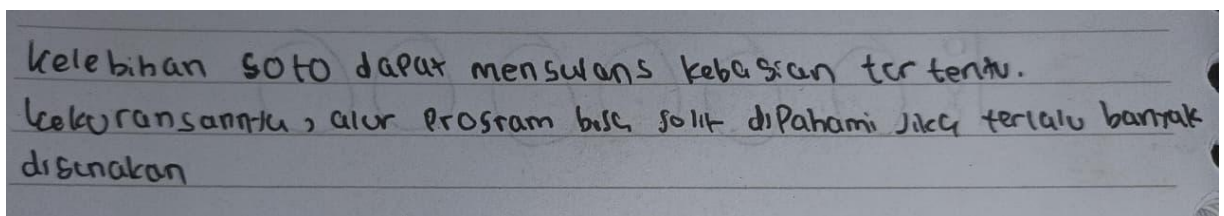
Jawab **8 pertanyaan** yang terdapat pada modul:

1. Apa perbedaan utama antara if dan if-else?

if hanya menjalankan proses saat kondisi benar, sedangkan if-else punya pilihan saat kondisi benar atau salah.

2. Kapan struktur else-if lebih sesuai digunakan daripada beberapa if yang berdiri sendiri?
else-if digunakan saat ada beberapa kondisi yang saling berkaitan dan hanya satu yang dijalankan.
3. Mengapa operator && menyebabkan kedua kondisi harus terpenuhi?
Karena && bernilai benar hanya jika kedua kondisi benar.
4. Apa fungsi { } pada compound statement?
{ } mengelompokkan beberapa statement menjadi satu blok.
5. Apa fungsi break pada switch-case?
break menghentikan case agar tidak lanjut ke case berikutnya.
6. Apa yang terjadi jika break dihilangkan dari sebuah case?
Program akan lanjut menjalankan case berikutnya sampai menemukan break.
7. Mengapa pembagian dengan nol perlu ditangani secara khusus?
Karena pembagian dengan nol tidak diperbolehkan dan dapat menyebabkan kesalahan pada program.
8. Apa kelebihan dan kekurangan penggunaan goto pada contoh percobaan?

Kelebihan goto dapat mengulang ke bagian tertentu. Kekurangannya, alur program bisa sulit dipahami jika terlalu banyak digunakan.



kelebihan goto dapat mengulang ke bagian tertentu.
kekurangannya, alur program bisa sulit dipahami jika terlalu banyak digunakan

H. TUGAS PRAKTIKUM

4.7.1 Bilangan ganjil atau genap

Analisa : Program menerima satu bilangan bulat dan menentukan apakah bilangan itu ganjil atau genap. Bilangan genap habis dibagi 2 (sisa bagi 0), sedangkan ganjil tidak.

Algoritma:

1. Baca bilangan.
2. Hitung $\text{bilangan} \% 2$.
3. Jika hasilnya 0, cetak "bilangan genap".
4. Jika tidak, cetak "bilangan ganjil".

```
#include <stdio.h>

int main() {
    int bilangan;

    printf("Masukkan angka = ");
    scanf("%d", &bilangan);

    if(bilangan % 2 == 0) {
        printf("bilangan genap");
    } else {
        printf("bilangan ganjil");
    }

    return 0;
}
```

Masukkan angka = 10
bilangan genap

Masukkan angka = 5
bilangan ganjil

Masukkan angka = 0
bilangan genap

no	input	Hasil yang diharapkan	Hasil aktual	keterangan
1	10	Bilangan genap	Bilangan genap	Sesuai
2	5	Bilangan ganjil	Bilangan ganjil	sesuai
3	0	Bilangan genap	Bilangan genap	sesuai

Analisis Hasil: Program benar untuk bilangan positif, nol, dan negatif. Bilangan negatif tetap benar karena kondisi yang dicek adalah $== 0$, bukan $== 1$.

4.7.2 Klasifikasi Karakter

Program mengklasifikasikan satu karakter menjadi huruf kecil, huruf besar, angka, atau karakter khusus, dengan memanfaatkan urutan kode ASCII.

Algoritma:

1. Baca karakter (spasi sebelum %c membuang whitespace sisa).
2. Jika 'a' <= karakter <= 'z', cetak "Huruf kecil".
3. Jika tidak, jika 'A' <= karakter <= 'Z', cetak "Huruf besar".
4. Jika tidak, jika '0' <= karakter <= '9', cetak "Angka".
5. Selain itu, cetak "Karakter khusus".

```
#include <stdio.h>

int main() {
    char karakter;

    printf("Masukkan karakter = ");
    scanf(" %c", &karakter);

    if (karakter >= 'a' && karakter <= 'z') {
        printf("Huruf kecil");
    } else if (karakter >= 'A' && karakter <= 'Z') {
        printf("Huruf besar");
    } else if (karakter >= '0' && karakter <= '9') {
        printf("Angka");
    } else {
        printf("Karakter khusus");
    }

    return 0;
}
```

Masukkan karakter = a	Masukkan karakter = =
Huruf kecil	Karakter khusus

Masukkan karakter = A	Masukkan karakter = 55
Huruf besar	Angka

No	Input	Hasil yang diharapkan	Hasil aktual	Keterangan
1	a	Huruf kecil	Huruf kecil	Sesuai
2	A	Huruf besar	Huruf besar	Sesuai
3	=	Karakter khusus	Karakter khusus	sesuai
4	55	angka	angka	sesuai

Analisis Hasil: Semua kategori terklasifikasi dengan benar. Program hanya membaca satu karakter, jadi input seperti "abc" hanya diproses huruf pertamanya.

4.7.3 Kalkulator sederhana

Program berupa kalkulator berbasis *accumulator*. Angka pertama disimpan di accumulator, lalu tiap operator dan angka berikutnya memperbarui nilainya. Operator yang didukung: + - * / % & |, s (tampilkan hasil), dan e (akhiri).

Algoritma :

1. Baca accumulator dan operator pertama.
2. Selama operator bukan e:
 - o Gunakan `switch` untuk memilih operasi.
 - o Untuk operator aritmatika/bitwise, baca bilangan lalu perbarui accumulator dan tampilkan hasilnya.
 - o Untuk /, jika bilangan 0 tampilkan error dan hentikan program.
 - o Jika operator tidak dikenal, cetak "Operator tidak valid".
 - o Baca operator berikutnya.
3. Cetak "Akhir perhitungan".

```
#include <stdio.h>

int main() {
    int accumulator, bilangan;
    char op;

    printf("Mulai perhitungan\n");
    scanf("%d %c", &accumulator, &op);

    while (op != 'e') {
        switch (op) {
            case 's':
                printf("= %d\n", accumulator);
                break;

            case '+':
                scanf("%d", &bilangan);
                accumulator += bilangan;
                printf("= %d\n", accumulator);
                break;
```

```

    case '-':
        scanf("%d", &bilangan);
        accumulator -= bilangan;
        printf("= %d\n", accumulator);
        break;

    case '*':
        scanf("%d", &bilangan);
        accumulator *= bilangan;
        printf("= %d\n", accumulator);
        break;

    case '/':
        scanf("%d", &bilangan);

        if (bilangan == 0) {
            printf("Error: tidak boleh dibagi 0\n");
            return 1;
        }

        accumulator /= bilangan;
        printf("= %d\n", accumulator);
        break;

    case '%':
        scanf("%d", &bilangan);
        accumulator %= bilangan;
        printf("= %d\n", accumulator);
        break;

    case '&':
        scanf("%d", &bilangan);
        accumulator &= bilangan;
        printf("= %d\n", accumulator);
        break;

    case '|':
        scanf("%d", &bilangan);
        accumulator |= bilangan;
        printf("= %d\n", accumulator);
        break;

    default:
        printf("Operator tidak valid\n");
}

scanf(" %c", &op );

printf("Akhir perhitungan\n");

return 0;
}

```

```

Mulai perhitungan
10 s
= 10
+5
= 15
*2
= 30
s
= 30
e
Akhir perhitungan

```

```

Mulai perhitungan
20s
= 20
/4
= 5
-2
= 3
e
Akhir perhitungan

```

```

Mulai perhitungan
100 s
= 100
-50
= 50
/5
= 10
+5
= 15
e
Akhir perhitungan

```

No	input	Hasil yang diharapkan	Hasil aktual	keterangan
1	$10 + 5 = 15 * 2 = 30$	$10 + 5 = 15 * 2 = 30$	$10 + 5 = 15 * 2 = 30$	Sesuai
2	$20/4 = 5 - 2 = 3$	$20/4 = 5 - 2 = 3$	$20/4 = 5 - 2 = 3$	Sesuai
3	$100 - 50 = 50 / 5 = 10 + 5 = 15$	$100 - 50 = 50 / 5 = 10 + 5 = 15$	$100 - 50 = 50 / 5 = 10 + 5 = 15$	Sesuai

Analisis Hasil: Operasi dasar dan bitwise bekerja benar, dan pembagian nol ditangani.

4.7.4 Tahun Kabisat

Program menentukan apakah suatu tahun (rentang valid 1900-2400) adalah tahun kabisat. Sebuah tahun kabisat jika habis dibagi 400, atau habis dibagi 4 tetapi tidak habis dibagi 100.

Algoritma:

1. Baca tahun.
2. Jika tahun < 1900 atau > 2400 , tampilkan pesan di luar rentang.
3. Jika tidak, jika tahun $\% 400 == 0$ atau (tahun $\% 4 == 0$ dan tahun $\% 100 != 0$), cetak "Tahun kabisat".
4. Selain itu, cetak "Bukan tahun kabisat".

```
#include <stdio.h>

int main() {
    int tahun;

    printf("Masukkan tahun = ");
    scanf("%d", &tahun);

    if (tahun < 1900 || tahun > 2400) {
        printf("Tahun harus berada di antara 1900-2400");
    }
    else if (tahun % 400 == 0 ||
            (tahun % 4 == 0 && tahun % 100 != 0)) {
        printf("Tahun kabisat");
    }
    else {
        printf("Bukan tahun kabisat");
    }

    return 0;
}
```

```
Masukkan tahun = 2024 Masukkan tahun = 1900
Tahun kabisat          Bukan tahun kabisat
Masukkan tahun = 2000 Masukkan tahun = 2023
Tahun kabisat          Bukan tahun kabisat
```

```
Masukkan tahun = 1800
Tahun harus berada di antara 1900-2400
```

no	input	Hasil yang diharapkan	Hasil aktual	keterangan
1	2024	Tahun kabisat	Tahun kabisat	Habis dibagi 4, bukan kelipatan 100
2	1900	Bukan tahun kabisat	Bukan tahun kabisat	Kelipatan 100 tetapi bukan kelipatan 400
3	2000	Tahun kabisat	Tahun kabisat	Kelipatan 400
4	2023	Bukan tahun kabisat	Bukan tahun kabisat	Tidak habis dibagi 4
5	1800	Tahun harus berada di antara 1900-2400	Tahun harus berada di antara 1900-2400	Validasi rentang

Analisis Hasil: Logika operator ||, &&, dan != bekerja benar, termasuk kasus khusus 1900 dan 2000. Validasi rentang dicek lebih dulu agar tahun di luar batas tidak diproses.

4.7.5 Kategori nilai

Program mengubah nilai angka (0-100) menjadi kategori huruf. Rentang di bawah adalah asumsi saya, sesuaikan dengan modul.

Algoritma:

1. Baca nilai.
2. Jika nilai < 0 atau > 100 , tampilkan pesan tidak valid.
3. Jika tidak, periksa dari yang tertinggi: $\geq 80 \rightarrow A, \geq 70 \rightarrow B, \geq 60 \rightarrow C, \geq 50 \rightarrow D$, selain itu E.

```
#include <stdio.h>

int main() {

    int nilai;

    printf("Masukkan nilai = ");
    scanf("%d", &nilai);

    if (nilai < 0 || nilai > 100) {
        printf("Nilai harus berada di antara 0-100");
    } else if (nilai >= 80) {
        printf("Kategori A");
    } else if (nilai >= 70) {
        printf("Kategori B");
    } else if (nilai >= 60) {
        printf("Kategori C");
    } else if (nilai >= 50) {
        printf("Kategori D");
    } else {
        printf("Kategori E");
    }

    return 0;
}
```

Masukkan nilai = 85
Kategori A

Masukkan nilai = 74
Kategori B

Masukkan nilai = 64
Kategori C

Masukkan nilai = 50
Kategori D

Masukkan nilai = 49
Kategori E

Masukkan nilai = 105
Nilai harus berada di antara 0-100

no	input	Hasil yang diharapkan	Hasil aktual	Keterangan
1	85	A	A	Sesuai
2	74	B	B	Sesuai
3	64	C	C	Sesuai
4	50	D	D	Sesuai
5	49	E	E	Sesuai

6	105	Nilai harus berada di antara 0-100	Nilai harus berada di antara 0-100	Validasi input
---	-----	------------------------------------	------------------------------------	----------------

Analisis Hasil : Struktur else-if mengevaluasi kondisi dari atas ke bawah dan berhenti di kondisi pertama yang benar. Karena itu cukup menulis batas bawah (≥ 80 , ≥ 70 , dst.) tanpa batas atas. Urutan kondisi sangat penting: kalau ≥ 50 ditaruh paling atas, semua nilai ≥ 50 akan masuk D.

4.7.6 Akar persamaan kuadrat

Program mencari akar persamaan $Ax^2 + Bx + C = 0$ memakai rumus kuadrat. Jenis akar ditentukan oleh determinan $D = B^2 - 4AC$:

- $D > 0$: dua akar real berbeda.
- $D = 0$: dua akar real kembar.
- $D < 0$: dua akar kompleks.

Algoritma:

1. Baca A, B, C.
2. Hitung $D = B^2 - 4AC$ dan cetak.
3. Jika $D > 0$, hitung $x_1 = (-B + \sqrt{D})/2A$ dan $x_2 = (-B - \sqrt{D})/2A$.
4. Jika $D = 0$, hitung $x_1 = x_2 = -B/2A$.
5. Jika $D < 0$, bagian real $= -B/2A$ dan bagian imajiner $= \sqrt{(-D)}/2A$, lalu cetak $x_1 = \text{real} + \text{imajiner} \cdot i$ dan $x_2 = \text{real} - \text{imajiner} \cdot i$.

```

#include <stdio.h>
#include <math.h>

int main() {

    float A, B, C;
    float D, x1, x2;

    printf("Masukkan nilai A = ");
    scanf("%f", &A);

    printf("Masukkan nilai B = ");
    scanf("%f", &B);

    printf("Masukkan nilai C = ");
    scanf("%f", &C);

    D = (B * B) - (4 * A * C);

    printf("Determinan (D) = %.2f\n", D);

    if (D > 0) {

        x1 = (-B + sqrt(D)) / (2 * A);
        x2 = (-B - sqrt(D)) / (2 * A);

        printf("x1 = %.2f\n", x1);
        printf("x2 = %.2f\n", x2);

    } else if (D == 0) {

        x1 = -B / (2 * A);

        printf("x1 = x2 = %.2f\n", x1);

    } else {

        float bagianReal;
        float bagianImajiner;

        bagianReal = -B / (2 * A);
        bagianImajiner = sqrt(-D) / (2 * A);

        printf("x1 = %.2f + %.2fi\n", bagianReal, bagianImajiner);
        printf("x2 = %.2f - %.2fi\n", bagianReal, bagianImajiner);

    }

    return 0;
}

```

```
Masukkan nilai A = 1
Masukkan nilai B = -3
Masukkan nilai C = 2
Determinan (D) = 1.00
x1 = 2.00
x2 = 1.00
```

```
Masukkan nilai A = 1
Masukkan nilai B = 2
Masukkan nilai C = 1
Determinan (D) = 0.00
x1 = x2 = -1.00
```

```
Masukkan nilai A = 1
Masukkan nilai B = 2
Masukkan nilai C = 5
Determinan (D) = -16.00
x1 = -1.00 + 2.00i
x2 = -1.00 - 2.00i
```

```
Masukkan nilai A = 0
Masukkan nilai B = 2
Masukkan nilai C = 1
Determinan (D) = 4.00
x1 = -1.#J
x2 = -1.#J
```

no	input	Hasil yang diharapkan	Hasil aktual	keterangan
1	1,-3,2	D = 1.00; x1 = 2.00; x2 = 1.00	D = 1.00; x1 = 2.00; x2 = 1.00	Dua akar real berbeda
2	1,2,1	D = 0.00; x1 = x2 = -1.00	D = 0.00; x1 = x2 = -1.00	Akar kembar
3	1,2,5	D = -16.00; x1 = -1.00 + 2.00i; x2 = -1.00 - 2.00i	D = -16.00; x1 = -1.00 + 2.00i; x2 = -1.00 - 2.00i	Akar kompleks
4	0,2,1	Bukan persamaan kuadrat	x1 = nan, x2 = -inf	Program belum menangani A = 0

I. KESIMPULAN

1. Praktikum ini membantu memahami cara program mengambil keputusan berdasarkan kondisi yang diberikan melalui beberapa bentuk percabangan.
2. Penggunaan if, if-else, dan else-if dapat disesuaikan dengan kebutuhan program, mulai dari pemeriksaan satu kondisi hingga beberapa kemungkinan kondisi.
3. switch-case dapat digunakan untuk pemilihan berdasarkan nilai tertentu.
4. Penggunaan operator logika, compound statement, dan pengecekan kondisi khusus membantu program menangani masukan dengan lebih terarah, termasuk kondisi seperti pembagian dengan nol.

5. Pengujian dengan berbagai data diperlukan untuk memastikan program menghasilkan output sesuai kondisi yang diberikan.
-

J. CHECKLIST SEBELUM DIKUMPULKAN

No.	Komponen	✓
1	Halaman judul	✓
2	Tujuan praktikum	✓
3	Dasar teori	✓
4	Kode 7 percobaan	✓
5	Screenshot hasil running	✓
No.	Komponen	✓
6	Analisis setiap percobaan	✓
7	Tabel pengujian	✓
8	Jawaban 8 pertanyaan	✓
9	Kode dan hasil tugas	✓
10	Kesimpulan	✓